KG

PORTFOLIO

Kristopher Gallow

AI-native operator and systems builder. Senior Capital Underwriter with a Process Automation focus at Toast, plus four LLM-powered terminals shipped through Ghost Strategies LLC.

Live Site LinkedIn GitHub

7+
Years in financial services

12
Production tools shipped

$500M+
Capital deployments supported

FEATURED WORK

EA

Earnings Analysis AI Agent

Ghost Strategies · Autonomous LangGraph agent

Independent

A LangGraph state machine that behaves like an analyst, not a script: a router inspects each earnings release at run time and sends clean data down a free deterministic path while messy prose goes to the LLM. Every extracted figure is validated by real code — failures loop back with the errors injected into the retry prompt, capped at three attempts before failing loud to a human. Nothing ships without explicit approval; the graph pauses to ask. Live market data via yfinance, open source on GitHub.

Stack Python, LangGraph, LangChain, OpenRouter **Design** Code-validated · bounded retries · human-approved

CM

COO/COLE Case Management Dashboard

Toast

Toast · 10-person underwriting team · 700+ active cases

The team's 700+ row spreadsheet became a full application without leaving Google Workspace: a live case table with optimistic-UI edits, 15+ chart analytics including a cohort-bucketed Bizops funnel that separates queue-wait from rep turnaround, and a per-rep diagnostics panel that reconciles dashboard counts against raw sheet countifs. It now largely runs itself — nightly auto-expiry and monthly history snapshots on scheduled triggers, a Needs Attention queue for slipping cases, and no-code configuration so team leads administer everything from Settings without a developer in the loop.

Stack Apps Script, JavaScript, Tailwind, Chart.js (2 custom plugins) **Users** 10

UC

Underwriting & Collections Toolkit

Toast

Toast · Capital Underwriting & Collections

Four client-side tools sharing one parse/classify engine so results never diverge: a lien docket dashboard that de-dupes and type-classifies UCC & tax-lien filings with per-row badges, in-page bookmarklets running the identical engine against the logged-in data provider, a color-coded application-routing advisor, and a collections extractor that defeats an AG-Grid's row and column virtualization by stitching cells into whole rows by stable column ID — with a paste fallback that parses multi-line, JSON-valued cells. All in-browser, nothing transmitted; part of a toolset that cut review time ~60%.

Stack React, Vanilla JS, Bookmarklets **Impact** Part of a toolset that cut review time ~60%

SQ

Self-Serve Snowflake Query Helper

Toast

Toast · Capital Underwriting

Plain English in, Snowflake SQL out — with a teaching layer that maps each phrase of the request to the exact clause it became, so non-analysts learn while the ad-hoc-query bottleneck on senior analysts disappears. Under the hood: one normalized contract over nine LLM providers with live model discovery (new models appear with zero code changes), a single effort control translated per model family into three different reasoning mechanisms, and a token-gated, fail-closed proxy that sends only schema metadata — never a row of data. The same engine ships three ways: web app (including a fully local Ollama mode), Cowork skill, and a claude.ai artifact that bills the viewer's own subscription.

Stack Vanilla JS, Cloudflare Workers, 9-provider LLM proxy, Ollama **Forms** Web app · Cowork skill · Claude artifact

FV

Funding Verification Tool

Toast · Capital Underwriting · 20-35 minute workflow cut to 40 seconds

Toast

A single HTML file that replaced a fragile VLOOKUP chain: drag in four source files and a hash-join engine reconciles hundreds of thousands of rows in milliseconds across nine tabs, with processor-specific match logic isolated in code instead of formulas. Large bank files parse through a persistent three-worker pool with zero-copy transfers; exports edit a bundled template's XML at runtime so the master sheet's conditional formatting survives; and a TCA subsystem auto-detects the canonical GUID column by scoring UUID density against uniqueness. A 20–35 minute batch became 40 seconds.

Stack Vanilla JavaScript, SheetJS, custom RFC 4180 parser **Sources** 4-way JOIN across 9 tabs

GS

Ghost Strategies Terminal Suite

Ghost Strategies LLC · LLM-powered financial tooling

Independent

Six production tools built with Anthropic's Claude, plus the practice that runs on them: Wealth Compass (a 13-sheet, 755-formula workbook parsed entirely client-side — no data leaves the device), Training Terminal (real Python, SQL, JavaScript, and Rust execution in the browser, 200 curated questions over five gated tiers), Model Bench Terminal (blind LLM evaluation feeding a cost-vs-win-rate leaderboard that assigns the other tools' model slots), Economic Indicators Terminal (a macro dashboard whose research desk never lets the model produce a charted number or an unsourced fact), Garage Terminal (FastAPI asset tracking with scheduled CI/CD pipelines), and Ghost Lead Terminal (the CRM behind a governed, human-in-the-loop agent operating model).

Stack Python, FastAPI, JavaScript, Pyodide, sql.js

Discipline I own the logic, the LLM writes the code, I validate every output

CX

CSV Column Extractor

Toast · Bizops · 9-minute workflow cut to under 55 seconds

Toast

Performance engineering wrapped in a tool simple enough for non-technical operations users: streams 1M+ row spreadsheets through parallel Web Workers, with a pipeline that starts file I/O the moment a file lands and pre-warms workers by compiling SheetJS during the user's review window. The parser constructs only the three needed cell addresses per row instead of materializing full sheets, and clipboard output ships in 150K-row chunks calibrated to stay under Google Sheets' 10-million-cell limit. A nine-minute open-to-paste workflow now takes under 55 seconds.

Stack Vanilla JavaScript, Web Workers, Papa Parse, SheetJS, Promise.all **Scale** 1M+ rows/file

ET

Email Template Automation Suite

Toast

Toast · 14-person Capital Underwriting and Change of Ownership · 11 phases

Eleven phases from CLI script to native-feeling desktop app for a 14-person team: 20 pre-loaded templates that copy with rich formatting intact — via AppleScript's «class HTML» on macOS and hand-built CF_HTML byte-offset headers on Windows — cutting retrieval from 30 seconds to under 3. Deliberately shipped as a 15KB Flask app instead of a 200MB Electron bundle, with zero AI tokens consumed and zero customer data exposure by design: placeholders fill locally, so nothing ever leaves the machine. Grew into a Salesforce-embedded generator concept presented to engineering stakeholders.

Stack Python, Flask, PowerShell, AppleScript, PyInstaller, pystray **Platforms** macOS + Windows

T

TOAST INC · SENIOR CAPITAL UNDERWRITER, PROCESS AUTOMATION

Toast Work

Four production tools built for the Capital Underwriting and Change of Ownership teams. Each replaced a fragile or manual workflow with a self-contained, browser-based system.

COO/COLE Case Management Dashboard

Full-stack web app · 10-person underwriting team · 700+ active cases · self-running

A full-stack web application built on top of a live Google Sheet — no separate backend or hosting — that replaced manual spreadsheet workflows for case tracking, analytics, and address-change requests across a 10-person team and 700+ cases. The Cases tab tracks the full lifecycle with optimistic-UI edits written straight back to the sheet, the Address Change tab manages a parallel five-status workflow on its own sheet, and the Analytics tab is an operational scorecard with team-level, per-rep, and Bizops-funnel views — all scopeable by year from a single header filter. Over successive iterations the tool evolved from a case tracker into a largely self-running system: non-technical leads administer it with zero code changes, it runs its own scheduled maintenance, and it documents itself for whoever maintains it next.

MOCK VIEW

Case Management Dashboard · Cases (47 active · 3 picked today)

47	12	4.2	98%
Active Cases	Completed (7d)	Avg Days TTC	SLA Met

CASE #	RESTAURANT	SOURCE	STATUS	DAYS
C-1042	[Redacted]	Bizops	In Review	2
C-1041	[Redacted]	Collections	Picked	1
C-1040	[Redacted]	Direct	Completed	3
C-1039	[Redacted]	Bizops	In Review	5

KEY FEATURES

- Live case table backed by Google Sheets with an inline edit drawer, add/delete modals, and a manager Settings panel for self-serve config — plus a gated "Edit Case Details" mode that locks structural fields (Case #, Source, Type) behind an explicit toggle while status, rep, and notes stay instantly editable
- No-code administration: reps, submitters, statuses, case types, sources, address-change statuses, and the auto-expire/staleness thresholds all live in a Config tab edited through Settings — with self-healing hash-derived badge colors for newly added values, transparent schema backfills on upgrade, and off-list legacy values preserved (marked "legacy") rather than silently clobbered
- Scheduled autonomy: stale-case auto-expiry runs as a nightly time-driven trigger (moved out of the read path so reads stay write-free and cacheable), a monthly job appends summary

snapshots to an auto-created History tab for durable trend and audit history, and a "Needs Attention" tab surfaces open cases past a configurable staleness threshold with a live count badge

- Performance pass: page load collapsed from four serial RPCs into one bundled call with graceful fallback, and a short-TTL server-side analytics cache keyed per year and explicitly invalidated by every write — fast, without ever serving stale numbers
- Year filter: one header dropdown (YTD, All Time, or any year auto-discovered from the data) scopes both the case table and the full analytics engine, which rebases its year boundaries so past years render complete pictures; rolling 7/30-day lookbacks deliberately stay anchored to today and are labeled as such
- Analytics tab (30-day, MoM, YTD) with Time-to-Decision buckets and a Bizops/Collections funnel of three cohort-bucketed nested-bar charts (Created → Picked → Decided) that surface each month's open WIP as a visible band, separating queue-wait from rep turnaround
- Inherited-case days rebasing: for handoff cases, rep-time metrics are recomputed in-memory as (final date − pick date) so team and per-rep averages reflect rep-controlled time, not queue inflation — while system-time charts re-derive from raw dates and stay unaffected
- Rep Diagnostics panel: a per-rep deep dive (as submitter and as assigned rep) built to reconcile dashboard chart counts against sheet countifs, with row-level raw-value dumps that expose the hidden whitespace and casing that break exact-match counts
- Two custom Chart.js v4 plugins — barTotalsPlugin (per-bar and stacked totals) and nestedBarLabelsPlugin ("inner / outer" cohort labels) — with disciplined chart-instance teardown to avoid canvas-reuse leaks
- Hardened Sheets-as-database layer: a bottom-up last-row scan so appends never land at row 4,000+, LockService on concurrent writes (with try/finally lock release), VLOOKUP formula copy + flush before read-back, UTC date normalization, same-day 0-day clamping, and server-side status normalization — case-insensitive canonicalization against the config list — that eliminated an entire class of data-validation rejections
- Address Change tab on its own sheet — five-status lifecycle, full add-request parity (mark Complete and "Checked in Clear" at creation), idempotent status-column and data-validation auto-creation, and unified GUID search across both workflows
- Idle-refresh banner after 30 minutes of inactivity and a version-update notifier that polls the deployed app version every 10 minutes
- Self-documenting: an in-app About/Maintenance panel carries the deploy steps, scheduled-job reminders, and conventions a successor needs, and a Help modal explains every tab and automatic behavior — the institutional knowledge lives in the app, not in one person's head

STACK

`Google Apps Script` `HTML` `CSS` `JavaScript` `Tailwind CSS` `Chart.js v4` `Google Sheets` `google.script.run` `LockService` `CacheService` `Time-driven triggers` `Google OAuth`

Email Template Automation Suite

11 phases · cross-platform desktop app · 14-person team · 30s to under 3s

A cross-platform desktop application that ships 20 pre-loaded email templates with rich-text formatting preserved on paste. Grew over 11 phases from a CLI tool to a full Flask web app, then to a PyInstaller-bundled native application with a system tray icon, and finally into a Salesforce-embedded NOIA generator concept presented to engineering stakeholders.

MOCK VIEW

Email Templates · 20 templates · 14-person team

Capital Underwriting	Received Response · Template ready · ● Copied
NOIA	NOIA MCA · {date} · {restaurant_name} · {source_of_debt}
Change of Ownership	COO DS Sent · {date} · {restaurant_name}

Retrieval: avg 2.1s · Tokens consumed: 0 · Customer data sent to model: 0

PHASE PROGRESSION

- Phases 1 to 3: CLI tool, cross-platform GUI launcher, then a full Flask web app
- Phase 4: Dark mode, brand logo, and accordion drawer system for the editor
- Phase 5: PyInstaller-bundled standalone executable with platform-appropriate data directories (macOS app support, Windows AppData), port discovery to dodge AirPlay on macOS, and a system tray icon for clean exits
- Phase 6: Salesforce-embedded NOIA template generator concept, with function-based template rendering (eight templates, conditional logic for variable-length entity lists)
- Phases 7 to 9: Placeholder fill modal with confirmation toasts, multi-line field support, two-panel preview modal, and product demo enhancements
- Phase 10: Template-sync bug fix (silent failure mode on every launch caught and resolved)
- Phase 11: Stakeholder presentation HTML with Apps Script architecture diagrams and a NOIA automation proposal for engineering review

ENGINEERING DECISIONS

- Zero AI tokens consumed by design: templates use placeholders that the user fills locally, so no customer data ever leaves the machine
- Cross-platform clipboard formatting: macOS uses AppleScript with the «class HTML» format; Windows uses CF_HTML byte-offset headers with precise UTF-8 byte counting
- 15KB Flask app deliverable instead of a 150 to 200MB Electron bundle, deliberately avoiding the Apple Developer Program signing requirement

- Per-machine identity: no authentication needed, each install is local-only

STACK

Python Flask Bash PowerShell JavaScript AppleScript Quill.js PyInstaller pystray Pillow

Funding Verification Tool

Single-file HTML · 20-35 minute workflow cut to 40 seconds

A single-file HTML application that replaced a fragile VLOOKUP chain in Google Sheets used to verify merchant banking data for capital deployments. Performs an in-browser JOIN across four sources (New Funding file, Snowflake export, Original CSV funding records, and DSL bank files), produces match results with processor-specific logic, supports real-time manual overrides, and exports a multi-sheet XLSX workbook from a bundled template that preserves the master sheet's conditional formatting. The tool grew to 9 tabs, including a TCA Check tab that auto-flags Toast Checking accounts and matches Slack-pasted GUIDs against an auto-detected canonical GUID column, and an External MCA Refi tab that tracks refi lineage between loans. The full funding verification workflow used to take 20 to 35 minutes per batch; the latest measured time (June 2026) clocked it at 40 seconds end-to-end.

BEFORE AND AFTER

BEFORE · 20-35 MINUTES

1. Manually maintain VLOOKUP chains across four Google Sheets
2. #N/A errors propagate when Snowflake data is delayed, no path to manual fix
3. No audit trail for manual SPA verifications or Toast Checking accounts
4. WORLDPAY and CHASE rules embedded in formulas, easily broken

AFTER · 40 SECONDS

1. Drag-drop four files, JOIN runs in milliseconds across 9 tabs
2. Manual Overrides tab surfaces Check Data rows, recompute on entry
3. Toast Checking and Verified in Merchant labels persist across re-uploads
4. Processor logic in code, 4-sheet XLSX export preserves master sheet formatting

MOCK VIEW

Funding Verification · Ops SPA Deals (12 rows)

EXT ID	PROC	MATCH R	MATCH A	LABEL
EXT-8821	WORLDPAY	YES	YES	Toast Checking
EXT-8820	CHASE	YES	Check Data	Pending
EXT-8819	WORLDPAY	NO	YES	Verified
EXT-8818	CHASE	YES	YES	Verified

KEY ENGINEERING DECISIONS

- Custom RFC 4180 CSV parser written from scratch to replace SheetJS's unreliable CSV mode (BOM stripping, mixed line endings, embedded quotes)
- MID normalization for JavaScript scientific notation: Snowflake XLSX exports return Merchant IDs as 1.23E+9, breaking direct string compares
- Hash-join via Map objects: equivalent to NewFunding LEFT JOIN Snowflake LEFT JOIN OriginalCsv LEFT JOIN DSL, executes in milliseconds for hundreds of thousands of rows
- Persistent DSL Worker Pool (size 3): operators routinely drop 4 to 10 large DSL files at once, so XLSX parsing runs through a fixed pool of three Web Workers fed by a queue and dispatcher, with zero-copy ArrayBuffer transfer and workers reused across drops rather than re-spawned
- Processor-specific match logic isolated in code: WORLDPAY requires exact DSL-to-file match; CHASE requires file value to differ from a known placeholder
- Real-time override injection: when a Snowflake lookup misses, manual GUID/MID/Proc Partner entry constructs a synthetic Snowflake-like object and the JOIN re-runs without a page refresh
- Labels and overrides keyed to External ID, survive re-uploads of any source file within the same session
- Template-based XLSX export: bundles the master Google Sheet's exact conditional formatting rules in a base64 template and edits the template XML at runtime via JSZip to inject the live data, preserving every YES/NO/Check Data cell fill from the original
- TCA Check subsystem: auto-detects the canonical Toast GUID column per source using a UUID-density times uniqueness score, auto-flags Toast Checking accounts from the Original CSV, and matches Slack-pasted GUIDs against in-batch records with per-loan and per-merchant dedup modes
- Refi lineage tracking: the External MCA Refi tab links each refi loan to its original (matching the External ID minus its suffix) and highlights the original loans magenta in the XLSX export
- Multi-sheet audit export: one workbook with the Ops sheet plus additional audit sheets, each filename-stamped MMDDYYYY-Funding Verification Tool.xlsx
- 9-tab structure: Ops SPA Deals (primary output with stat-pill filter shortcuts), four source upload tabs, TCA Check, Refi, External MCA Refi, and Manual Overrides
- Ships with two companion artifacts: a 13-page Standard Operating Procedure (built with a python-docx script, 11 embedded screenshots, data-flow diagram, troubleshooting guide, and a domain glossary), and a standalone Funding File Validator that round-trip-checks the exported file against the source before handoff

STACK

`Vanilla JavaScript` `SheetJS 0.18.5` `JSZip 3.10.1` `Custom RFC 4180 parser` `Web Worker pool` `Web FileReader API` `Base64 template embedding` `Clipboard API`

CSV Column Extractor

Single-file HTML · 9-minute workflow cut to under 55 seconds

A single-file HTML dashboard for combining up to five large spreadsheet files, each potentially exceeding one million rows, and extracting three specific columns for paste into a destination Google Sheet. Iterative performance engineering took the full open-to-paste workflow on three 25MB XLSX files from approximately nine minutes (manual baseline) to under 55 seconds.

BEFORE AND AFTER

BEFORE · ~9 MINUTES

1. Open three files sequentially in Excel or Sheets
2. Manually select and copy three columns from each file
3. Paste each chunk into the destination sheet, watch for cell-limit failures
4. Retry failed pastes after Google Sheets silently truncates

AFTER · UNDER 55 SECONDS

1. Drag-drop all three files into the tool
2. Pre-read and pre-warm pipeline starts in the background
3. Auto-parse fires 400ms after the last file, all three parse in parallel
4. Copy each column with one click, live cell-limit warnings prevent silent failures

MOCK VIEW

CSV Column Extractor · 3 files queued · auto-parse in 0.4s

847,302	3	~22s	3.9M
Total Rows	Files Parsed	Parse Time	Cells / Chunk

COLUMN	ROWS	CELL ESTIMATE	ACTION
Column B (MID)	847,302	22.0M cells	Chunk Required
Column L (R&T)	847,302	22.0M cells	Chunk Required
Column M (DDA)	847,302	22.0M cells	Chunk Required

Chunk size 150K rows · 6 chunks per column · safely under 10M cell limit

PERFORMANCE PIPELINE

- Parallel parsing via Promise.all: all files parse simultaneously in independent workers with local accumulators, merged in original file order after all Promises resolve

- Pre-read FileReader cache: readAsArrayBuffer fires the moment a file enters the queue, not when Parse is clicked, eliminating I/O wait from the critical path
- Pre-warmed worker pool: a Web Worker is spawned per XLSX file on drop, compiling SheetJS (about 500KB) inside the worker during the user's file-review window
- Auto-parse with 400ms debounce: no Parse button click required, handles single-file and multi-file drops uniformly, guards against re-triggering once results are displayed
- Selective XLSX cell extraction: the worker constructs only three required cell addresses per row (B+n, L+n, M+n) instead of allocating the full 2D array via sheet_to_json
- Batch and chunk sizes increased 4x: worker message batches went from 25K to 100K rows; CSV chunk size went from 5MB to 20MB, reducing postMessage round-trips proportionally

BASE ENGINEERING DECISIONS

- Papa Parse with worker: true for CSV streaming; hand-written Blob Worker for SheetJS XLSX parsing to keep the UI responsive during 1M+ row binary parses
- ArrayBuffer transferred via postMessage transferables to eliminate the memory cost of a second copy on large files
- 150K-row clipboard chunks: 150K rows times 26 default Google Sheets columns equals 3.9M cells, safely under the 10M limit
- Push-in-loop array construction instead of spread or push.apply, avoiding call-stack overflow on arrays exceeding 125K elements
- Three primary copy buttons (one per column) because the destination sheet uses non-adjacent columns; combined tab-separated copy preserved as a secondary option

STACK

`Vanilla JavaScript` `Web Workers` `Papa Parse 5.4.1` `SheetJS 0.18.5` `Blob Workers` `Promise.all parallelism` `FileReader API` `Clipboard API`

Underwriting & Collections Toolkit

Client-side underwriting & collections tools · React + vanilla-JS bookmarklets

A suite of zero-dependency, browser-based tools that replaced brittle spreadsheet macros across lien review, application triage, and collections history. Analysts paste or scrape third-party lien/UCC records (from a KYB data provider) and application details and get parsed, de-duplicated, recency-filtered, type-classified output — or a color-coded routing recommendation — to drop straight into notes. The same shared-engine pattern was extended to collections, extracting parent-account default and churn history from a virtualized data grid into a standardized, paste-ready summary. Everything runs locally, no backend, and transmits no data. Part of a set of underwriting tools that together cut review time by roughly 60%.

WHAT'S UNIQUE

- Lien Docket Dashboard: ingests pasted card text or raw API JSON, extracts per-filing fields, de-dupes, sorts newest-first, applies recency windows, and tags each filing by type with per-row badges so misclassification is obvious at a glance; summary metrics, diagnostics, one-click copy, and .txt/.csv export
- Two decision-support bookmarklets: a lien scraper that runs the identical parse/classify engine in-page against the logged-in provider (sidestepping cross-origin and authentication limits), and an App Router that reads live application fields and renders a color-coded review-depth recommendation
- Parent Default & Churn Extractor: reads parent-account default and churn history from a virtualized AG-Grid tracker into a standardized, paste-ready summary — defeating row- and column-virtualization by stitching cells into whole rows by stable column ID (not screen position), with a paste fallback that record-splits multi-line, JSON-valued cells; the grid was reverse-engineered first with a read-only DOM probe
- One shared parse/classify/format engine across every tool, so results never diverge; fail-loud by design — unreadable dates are kept and flagged, never silently dropped
- Config over hard-coding: keyword lists, recency windows, routing thresholds, and field labels live in editable config blocks so rule changes don't touch the logic

STACK

`React 18` `Babel Standalone` `Vanilla JavaScript` `Bookmarklets` `AG-Grid extraction` `Clipboard API` `Blob/URL export`

Self-Serve Snowflake Query Helper

Teaching-oriented text-to-SQL · web app + Worker proxy · Cowork skill · Claude artifact

A browser tool that lets non-analyst teammates draft accurate, schema-aware Snowflake queries in plain English — and learn the SQL as they go, through an explicit phrase-to-clause mapping. It removes the bottleneck of senior analysts writing every ad-hoc query while keeping execution in the user's own Snowflake session. By design it never connects to Snowflake and never sees a row of data: it handles only schema metadata (table and column names, types, comments) plus the user's generic natural-language ask. All model calls route through a Cloudflare Worker that holds provider keys server-side and normalizes nine LLM providers behind one contract. The same schema-plus-teaching engine ships in three forms: the full web app, a Cowork skill that runs on the team's existing Claude seats, and a claude.ai artifact that runs on the viewer's own subscription — the last two with no API keys or infrastructure at all.

WHAT'S UNIQUE

- Multi-provider proxy: one normalized request/response contract over nine LLM providers (Anthropic, OpenAI, Gemini, xAI, DeepSeek, Moonshot, Zhipu, Mistral, Qwen) via an adapter pattern — seven OpenAI-compatible plus dedicated Anthropic and Gemini adapters — resolving browser CORS and removing API keys from the client
- Personal or organizational: a first-run selector configures either an organizational mode (shared team Worker, provider keys as server-side secrets, gated by a Bearer token) or a personal mode — bring-your-own-key (Anthropic or Gemini directly from the browser), your own proxy for all nine providers, or fully local via Ollama with installed-model detection and nothing leaving the machine
- Zero-key distribution: the engine also ships as a Cowork skill (runs on existing Claude seats, with adversarially hardened guardrails — execution and file-write bans plus a non-waivable PII pre-flight) and as a claude.ai artifact powered by the artifact runtime's completion API, billing the viewer's own subscription with zero infrastructure
- Self-updating model discovery: queries each provider's live model endpoint and merges it with a curated overlay (24-hour TTL), so newly released models appear automatically with no code edits — built on the insight that model names churn but API families stay stable
- One Low/Medium/High effort control mapped per model family to three distinct mechanisms (reasoning-effort enum, thinking-token budget, or omitted), and auto-disabled when the selected model can't reason
- Teaching layer: every result returns the SQL plus a plain-English explanation and a three-column phrase → clause → why mapping table, reframing a query generator into a learning tool
- Compliance-driven design: no database connection, metadata-only prompts, provider keys held as server-side secrets behind a token-gated, fail-closed proxy, and a client-side PII/MNPI pre-flight that warns on emails, phone numbers, GUIDs, and account-ID-shaped strings

STACK

`Vanilla JavaScript` `Cloudflare Workers` `Tailwind (CDN)` `Prism.js` `localStorage`

9-provider LLM proxy

Ollama

React (artifact)

GS

GHOST STRATEGIES LLC · INDEPENDENT

Ghost Strategies

Six LLM-powered tools built independently — and a practice that runs on them, under a governed multi-agent operating model. Every project ships with the same discipline: I own the logic and architecture, the LLM writes the code, I validate every output, and I only prompt with test or sample data so no proprietary information ever enters the model.

Wealth Compass Terminal

Client-side financial planning dashboard

A client-facing financial planning dashboard that parses a 13-sheet workbook with 755 formulas entirely client-side. All data stays on the user's device. No backend, no data transmission, no model exposure to client information.

STACK

`JavaScript` `HTML` `SheetJS` `Custom formula engine`

Training Terminal

Gamified four-language coding terminal · Python, SQL, JavaScript, Rust

A single-file, browser-based coding terminal (xterm.js) for practicing Python, SQL, JavaScript, and Rust. Runs real Python through Pyodide, real SQL through sql.js against a seeded relational database, and grades JavaScript inside a network- and storage-stripped Web Worker sandbox; Rust is graded by a literal-preserving structural matcher that accepts equivalent syntax. 200 curated questions — 50 per track across five gated tiers (Introductory, Amateur, Intermediate, Experienced, Master), 10 questions per gate — with localStorage-persisted progression and live tier and engine-status KPIs.

WHAT'S UNIQUE

- Four grading paths in one static file: Pyodide (Python), sql.js on a seeded auto-rebuilt database (SQL), an isolated Web Worker with network and storage APIs removed (JavaScript), and a structural normalizer with a syntax-equivalence system (Rust)
- Real xterm.js REPL: a command bar (start, hint, concepts, cheatsheet, examples, schema, reset, export/import), multi-line editing, command history, and emacs-style key bindings
- Learning layer baked in — per-tier concepts auto-surface on first entry, a toggleable cheatsheet panel, worked examples, and per-question hints
- 200 questions reviewed and calibrated by concept and difficulty; 10-question tier gates force mastery before the next rank unlocks, all persisted with no backend

STACK

`Vanilla JavaScript` `xterm.js` `Pyodide` `sql.js` `Web Workers` `localStorage`

Garage Terminal

Multi-asset tracker · Python FastAPI backend

A personal asset tracking terminal with a Python FastAPI backend and an automated data pipeline using GitHub Actions for weekly serverless CI/CD workflows. Pulls valuation data, refreshes the dashboard, and exposes a clean read API.

STACK

`Python` `FastAPI` `GitHub Actions` `JavaScript` `Serverless CI/CD`

Economic Indicators Terminal

Bloomberg-style macro dashboard · grounded LLM research desk · single HTML file

A dark, data-dense economic terminal with seven views — CPI and PPI deep dives, employment, projections with confidence bands, a cross-indicator correlation matrix, and a News & Analysis research desk — delivered as one ~3,500-line HTML file. It boots instantly from embedded data and upgrades itself to live FRED/BEA feeds when API keys are present, degrading gracefully: no keys, no network, and it still renders. The research desk is governed by one architectural law: *the model never produces a number that gets charted, and never states a fact without a source.*

WHAT'S UNIQUE

- Grounded LLM pipeline: a cheap router model classifies each question into a structured plan, deterministic JavaScript fetches the prices and computes every statistic (returns, CAGR, volatility, drawdowns, divergence windows), and a web-grounded narrator explains the exact date windows the math found — cited, streamed token-by-token, with the chart shading those windows
- Tiered model routing by query difficulty — fast, standard, or frontier narrator chosen per question; every answer names its model and shows its cost (typical queries under five cents)
- Provider fallback with honesty guarantees: three price providers behind one interface, failures named in the UI with reasons; mismatched price-adjustment bases refuse pair statistics rather than chart a distorted comparison
- IndexedDB caching with a 20-hour TTL makes repeat research free; the offline shell explains its state instead of erroring
- Security as a first-class constraint: keys runtime-only in localStorage with a pre-push scan, escape-first markdown rendering with scheme-validated source links, and prompt-injection resistance instructed in both system prompts
- 23-case embedded test suite against hand-verified fixtures, re-runnable in production; a formal adversarial audit of the data layer caught an XSS vector and a silent stats-corruption bug before anything was built on top of it

STACK

`Vanilla JavaScript` `Chart.js 4` `IndexedDB` `OpenRouter (multi-model)` `FRED / BEA APIs` `3 price providers` `Python CORS proxy` `Cloudflare Workers`

Model Bench Terminal

Blind LLM evaluation bench · model selection as recorded evidence

A single-file evaluation bench that turns model selection from vibes into recorded evidence. One prompt broadcasts to 2–4 models chosen from a live catalog of ~340; responses stream side by side with live time-to-first-token, throughput, and cost; quality ratings are collected *blind* (panes shuffled and anonymized until the rating locks); and every run writes a schema-versioned ledger row to IndexedDB, accumulating a private leaderboard of win rates by task tag with a cost-vs-win-rate scatter. The models that earn default slots in the other Ghost tools now earn them on the record.

WHAT'S UNIQUE

- Blind integrity treated as a security property, not a UI feature: every channel that leaks model identity — control-rail slugs, provider error text, history markers — is masked until reveal (blanked, not blurred, which survives select-copy)
- A cost chain that refuses to guess silently: exact stream usage → generation-endpoint lookup (captures web-search fees and pre-abort billing) → catalog estimate visibly tilde-marked; unknown costs render as "—", never a confident $0.0000
- Ledger schema designed to be frozen: schema version and UUID from day one, per-pane finish reason and serving provider captured because they can't be backfilled, and win-rate semantics pinned before any data accumulated
- Structured-output mode validates each response against an attached JSON schema client-side (a ~60-line purpose-built validator instead of a 120KB dependency) — how router-stage candidates for the other tools get vetted
- Hand-rolled SSE streaming with a per-pane AbortController and Promise.allSettled isolation, so one pane's mid-stream network death can't block or corrupt its siblings; escape-first rendering keeps hostile completions inert
- Audited and tested before first real use: independent code audit (15 findings resolved, including three schema gaps that would have been unrecoverable after data accumulated), headless end-to-end simulation against a stubbed API, and adversarial unit tests for the validator and renderer

STACK

`Vanilla JavaScript` `OpenRouter API` `IndexedDB` `Chart.js` `SSE streaming` `Zero build step`

Earnings Analysis AI Agent

Autonomous LangGraph agent · validate in code, retry with feedback, human-approve · github.com/KaeJhee/earnings-agent

An autonomous "earnings analyst" built as a self-directed project (2026): give it a ticker and it fetches live earnings data, decides on its own how to parse it, extracts revenue and EPS, validates the numbers with deterministic code, retries on failure with the errors fed into the next prompt, and pauses for human approval before shipping a beat/miss summary. It demonstrates the part of AI engineering that matters in production — not calling a model, but making its output trustworthy. Being rolled into the portfolio-analysis platform as its earnings-analysis module, scoped to public earnings data only.

```
START → fetch → route_by_shape → (format_fast | extract) → validate ⟲ → human checkpoint → synthesize → END
```

WHAT'S UNIQUE

- Self-correcting, not one-shot: deterministic validation checks every extracted figure, and failures loop back to the extractor with the errors injected into the retry prompt — capped at 3 attempts, then it fails loud to a human instead of shipping a wrong number
- The model generates; code judges — the AI never grades its own homework
- Human-in-the-loop by construction: the graph pauses (interrupt_before) and shows the numbers for approval; rejecting ships nothing
- Chooses its own path: a router inspects the data shape at run time — clean structured data takes a free deterministic path, messy prose goes to the LLM — the difference between a fixed script and an agent
- Production-shaped: refactored from a Jupyter prototype into a clean multi-file Python project (state, nodes, graph, CLI) with live market data via yfinance; the portfolio-terminal integration runs the reasoning model through OpenRouter, keeping it multi-model swappable without touching the graph
- Self-directed and documented: authored the build plan, a full written case study, and an interactive architecture walkthrough — all shipped in the repo

STACK

`Python` `LangGraph` `LangChain` `Anthropic Claude` `OpenRouter` `yfinance` `Human-in-the-loop` `CLI`

Ghost Lead Terminal

Single-file CRM · pipeline, outreach, and a validated import gate

A single-file HTML CRM that runs the practice's client pipeline end to end: a kanban board by stage, a sortable table with a full record drawer, a Today view for due and stale follow-ups, and a dashboard with honest funnel math. Data lives in a synced JSON file — no backend, no vendor lock — with conflict-merge prompts when two devices disagree. Agent-produced lead batches can only enter through a validated import gate, and outreach ships through a template system with a Gmail handoff and an explicit send-confirmation step, so nothing leaves without a human action.

WHAT'S UNIQUE

- Validated import gate with dedupe: agent research lands as schema-checked batch files, reviewed and merged deliberately — agents never write to the live data file
- Sync-aware single file: JSON data file synced across devices with a conflict-merge prompt instead of silent last-write-wins
- Two-click outreach flow: template fill → Gmail handoff → explicit "Email Submitted" confirmation, keeping the human send gate permanent
- Source attribution built into the schema so inbound-channel funnel math stays honest
- 24-check smoke-test suite beside the tool; schema field names treated as an additive-only contract

STACK

`Vanilla JavaScript` `Single-file HTML` `Synced JSON store` `Smoke-test suite`

Agent-Operated Practice

Operating model · a one-person practice run on a governed agent fleet

Ghost Strategies runs its operations — research, drafting, content preparation, and tooling — on a fleet of AI agents spanning multiple runtimes (Claude Code and Cowork among them, including a self-hosted research agent), designed around one constraint: the owner's review minutes are the bottleneck, so every lane minimizes them rather than maximizing agent output. Total planned operating overhead: about 80 minutes a week.

GOVERNANCE RULES (THE POINT OF THE SYSTEM)

- A human stands between generation and the public: agents research, draft, and package — nothing sends, posts, or publishes without an explicit human action
- Zero client PII to any model, any lane, any purpose — agents work from public data and tokenized templates only
- Verification over volume: a small batch of sourced, cited leads beats a large unverifiable one, and every factual claim in content carries the same bar
- Agents write batch files and drafts only; the CRM's validated import gate and human review are the only doors into the system of record
- Budget discipline: agent lanes run on capped, monitored spend — overruns are fixed with output limits and model routing, not more money

STACK

`Claude Code` `Cowork` `Multi-runtime agents` `Cron scheduling` `Human-in-the-loop gates`

AI-Assisted Financial Modeling Suite

Dynamic financial engines built with Claude

A separate workstream under Ghost Strategies focused on financial modeling rather than terminal apps. Includes a probability-weighted valuation model and a $3M TRS retirement planner. I own the financial logic and architecture while directing Claude to generate the code. Every output is validated against standard financial formulas before being trusted, and I only prompt with test or sample data to keep any real client information out of the model.

KG

ABOUT

Background and operating model

How I work and what I'm looking for.

I'm a Senior Capital Underwriter at Toast, where I support over $500M in active capital deployments and operate inside a fintech that scaled from $100M to over $2B in annual volume during my tenure. My day-to-day is capital decisions, customer health monitoring, and scenario planning, but my distinguishing work is building the systems and tools that operate alongside that core function. Ten of those tools are now in daily use across underwriting and collections, replacing manual workflows that used to consume hours of recurring team effort each week.

I'm part of Toast's Claude Code and Cowork rollout to a select group of process automation specialists. I built a multi-agent system with an orchestrator that routes work to specialized sub-agents (workflow automation, code audit, technical project management, devops, technical writing), each with its own system prompt, scoped tool access, and isolated context. The same architecture is replicated as Skills in Cowork.

Outside of Toast, I run Ghost Strategies LLC, where I ship LLM-powered financial tools. My operating discipline across every project is consistent: I own the logic and architecture, the LLM writes the code, I validate every output before I trust it, and I never feed proprietary data to a model.

I'm based in San Antonio, Texas, and I work remote-only. I'm targeting Senior Manager-track or Senior IC roles in strategy, operations, risk, or AI-process work at companies where the AI-native operating model is the default, not the exception.

CORE STACK

SQL (Snowflake) | Salesforce | Python | JavaScript | Google Apps Script | Hex | Excel and Google Sheets | Claude Code | Cowork | Multi-Agent Systems | Flask | FastAPI | Chart.js

KG

CONTACT

Get in touch

Best ways to reach me. I respond to email and LinkedIn within a business day.

DIRECT

EMAIL
krisgallow918@gmail.com

PHONE
832-233-9437

LINKEDIN
linkedin.com/in/kristophergallow

GITHUB
github.com/KaeJhee

LOCATION
San Antonio, TX (Remote)

SIDE PROJECT
kris@ghoststrategies.io

↓ Download resume (AI & Insights variant)